

```
SpannableString relative = new
SpannableString("This text is relatively small
or large");
relative.setSpan(new RelativeSizeSpan(1.5f),
relative.length() - 5, relative.length(), 0);
relative.setSpan(new RelativeSizeSpan(0.5f),
relative.length() - 14, relative.length() - 9,
0);
textView.setText(relative);
```

First, we create a `SpannableString` with the content. We then apply Spans with the method `SpannableString.setSpan()`. The first parameter in the method is the object of the type of Span that we want to apply; so we create a `RelativeSizeSpan` there. The parameter in the constructor of `RelativeSizeSpan` is the proportion or the multiplier.

For instance, passing 0.5 there would make the text half the size and passing 2 would make the text twice as big. The next two parameters in the `setSpan` method are the start and end of Span in the string. The last parameter is flag, which does not concern us now, so we will pass 0. This is how we can make text relatively small or big in a single `TextView`. If you want to apply some absolute size to your text, you can use `AbsoluteSizeSpan`, which applies the size to the text as absolute values.

BackgroundcolourSpan

This Span is used to apply a background colour to a piece of text. You ought to notice that we have used the term 'background colour', which means it only accepts a colour and not any other form of background such as `Drawables`. Now to apply this type of Span, use the following code:

```
SpannableString colour = new SpannableString("String with red
background");
colour.setSpan(new BackgroundcolourSpan(colour.RED), colour.
length() - "background".length(),
colour.length(), 0);
textView.setText(colour);
```

The parameter in the constructor of `BackgroundcolourSpan` is the colour you want as the background; the other parameters behave the same as explained earlier.

BulletSpan

`BulletSpan` can be used to apply bullet points to your strings. The usage is the same as in other Spans with one caveat. Let's look at the usage first.

```
SpannableString bullet = new SpannableString("String with
bullet point");
```

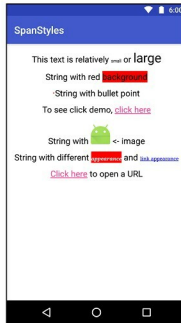


Figure 1: The different types of Spans

```
bullet.setSpan(new BulletSpan(5, colour.
RED), 0, bullet.length(), 0);
textView.setText(bullet);
```

The first parameter in the constructor of `BulletSpan` is the gap between the content and the bullet, and the other is the colour of the bullet. The caveat here is that in order to make `BulletSpan` draw a bullet point, it must be applied to the start of our string; otherwise, it won't draw the bullet.

Let's move on to some more interactive Spans.

ClickableSpan

`ClickableSpan` lets us make a certain part of the text clickable. It gives us the click event in an abstract method, `onClick`, that we need to implement. Let's see how we can apply it.

```
SpannableString clickable = new SpannableString("To see click
demo, click here");
clickable.setSpan(new ClickableSpan() {
@Override
public void onClick(View widget) {
Toast.makeText(widget.getContext(), "You just clicked
there", Toast.LENGTH_LONG)
.show();
}
}, clickable.length() - "click here".length(), clickable.
length(), 0);
textView.setText(clickable);
```

Here, we can see that we pass in an object of the abstract class `ClickableSpan` and implement the `onClick` method, which just makes a `Toast` to notify the click event. This might seem OK, but if you run the application and try to click on the link you just created, it will do nothing. This is because we explicitly need to tell `TextView` that the text may contain links. We do that by setting an object of the `LinkMovementMethod` in `TextView`.

```
textView.setMovementMethod(new LinkMovementMethod());
```

Now when you click on the link, it will show you a `Toast`.

ImageSpan

This Span is an interesting one, because we may end up in a situation where we need to show some kind of image in between our text. Now, what a novice Android developer would probably do is take an `ImageView` between two `TextViews`, as anyone who doesn't know anything about